BLOCK 7

EXPRESSIONS AND BASIC CALCULATIONS

Turning stored values into useful answers

```
unit_price = 12.50
quantity = 8
```

```
total_cost = unit_price
* quantity
```

```
print(total_cost)
```

```
100.0
```

30 min instruction • 30 min guided calculation lab

INSTRUCTION

Expression: a piece of code that produces a value

A value can be literal, named by a variable, or calculated from other values.

12.50	unit_price	unit_price * quantity	2 + 3
literal value	variable lookup	calculation	calculation

If Python can evaluate it to a value, it is acting like an expression.

INSTRUCTION

Arithmetic operators: Python calculator mode

Operators are symbols that tell Python what math operation to perform.

`+`	Addition 2 + 3 → 5
`-`	Subtraction 10 - 4 → 6
`*`	Multiplication 6 * 7 → 42
`/`	Division 7 / 2 → 3.5
`//`	Integer division 7 // 2 → 3
`%`	Remainder 7 % 2 → 1
`**`	Exponent 2 ** 3 → 8

Python uses * for multiply, not x.

INSTRUCTION

Three kinds of division show up a lot

Regular division, integer division, and remainders answer different questions.

```
7 / 2
3.5
exact-ish
division
```

```
7 // 2
3
whole groups
```

```
7 % 2
1
left over
```

Example mental model: 7 items split into groups of 2 gives 3 full groups with 1 left over.

INSTRUCTION

Order of operations: Python follows math rules

Parentheses make your intent obvious to both Python and humans.

```
2 + 3 * 4
→ 14
```

```
(2 + 3) * 4
→ 20
```

Parentheses

Exponents

Multiply / divide

Add / subtract

Left to right ties

When in doubt, add parentheses. Clear code beats clever code.

INSTRUCTION

Combining variables in calculations

Most real calculations use names, not raw numbers scattered everywhere.

```
unit_price = 12.50
quantity = 8

total_cost = unit_price *
quantity
```


Look up unit_price → 12.50

Look up quantity → 8

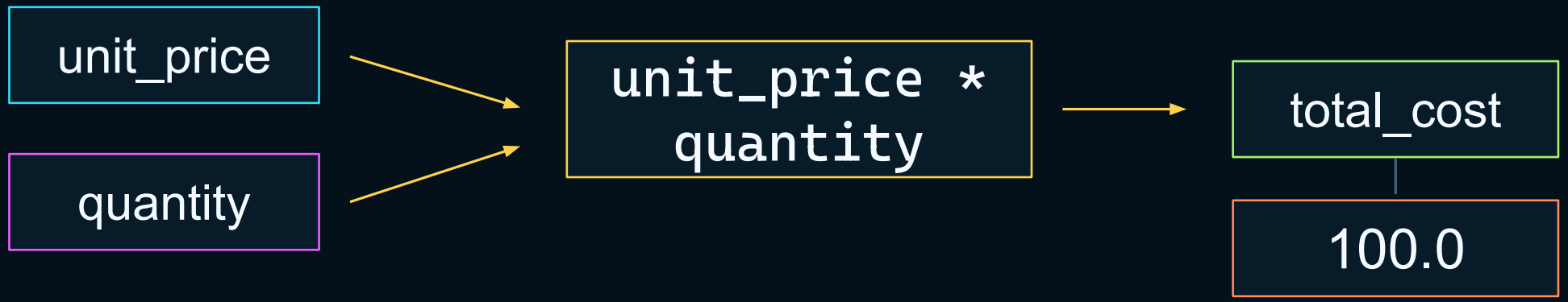
Multiply → 100.0

The expression on the right is evaluated first. Then the result is assigned to the variable name.

INSTRUCTION

Storing calculation results

A result variable lets later code reuse the answer without recalculating or copying it.




```
print("Total:", total_cost)
```

Output: Total: 100.0

INSTRUCTION

Common type mismatch: text plus number

Python will not always guess what you meant. That is a feature, not a bug.

```
total = 100
print("Total: " + total)
```

TypeError

```
total = 100
print("Total:", total)
```

```
total = 100
print("Total: " +
str(total))
```

Quick fix: use commas in print(), or explicitly convert with str().

INSTRUCTION

Same symbols, different meanings

Some operators work with multiple types, but the meaning depends on the types involved.

```
2 + 3
→ 5
```

```
"Py" + "thon"
→ Python
```

```
"ha" * 3
→ hahaha
```

number addition

text joining

text repetition

Types tell Python which version of an operation makes sense.

INSTRUCTION

Connection to processor instructions

A simple Python expression hides many lower-level steps.

```
total_cost = unit_price
* quantity
```

1. **Locate** the values behind the variable names
2. **Check** the value types and operation rules
3. **Perform** the multiplication using lower-level operations
4. **Store** the result and associate it with a name
5. **Display** the result as text on the screen if printed

Python line

human readable

many machine steps underneath

GUIDED EXERCISE

Guided calculation lab

Total cost

Average score

Percentage complete

Miles → kilometers

Minutes → hours + minutes

Bytes → KB / MB

Pattern: daefine variables → calculate result → print result → explain what happened.

GUIDED EXERCISE

Lab 1: total cost

Use multiplication to calculate the total price for several units.

```
unit_price = 12.50
quantity = 8

total_cost = unit_price *
quantity
print(total_cost)
```

Predict first

Run it

Change quantity to 12

Question: why is this better than typing 12.50 * 8 everywhere?

GUIDED EXERCISE

Lab 2: average score

Parentheses make the average calculation clear.

```
score1 = 88
score2 = 92
score3 = 79

average = (score1 + score2 +
score3) / 3
print(average)
```

Output:
86.33333333333333

Try without parentheses

Explain why it changes

Readable code tells the next person exactly what calculation you intended.

GUIDED EXERCISE

Lab 3: percentage complete

Percentages usually mean part divided by whole, then multiplied by 100.

```
completed_tasks = 18
total_tasks = 24

percent_complete =
completed_tasks / total_tasks
* 100
print(percent_complete)
```

18 ÷ 24 × 100

Output: 75.0

Meaning: 75 percent done

GUIDED EXERCISE

Lab 4: miles to kilometers

Unit conversions are just stored values plus a conversion factor.

```
miles = 5
km_per_mile = 1.60934

kilometers = miles *
km_per_mile
print(kilometers)
```


5 miles

8.0467 km

Good variable names can carry the units: miles, km_per_mile, kilometers.

GUIDED EXERCISE

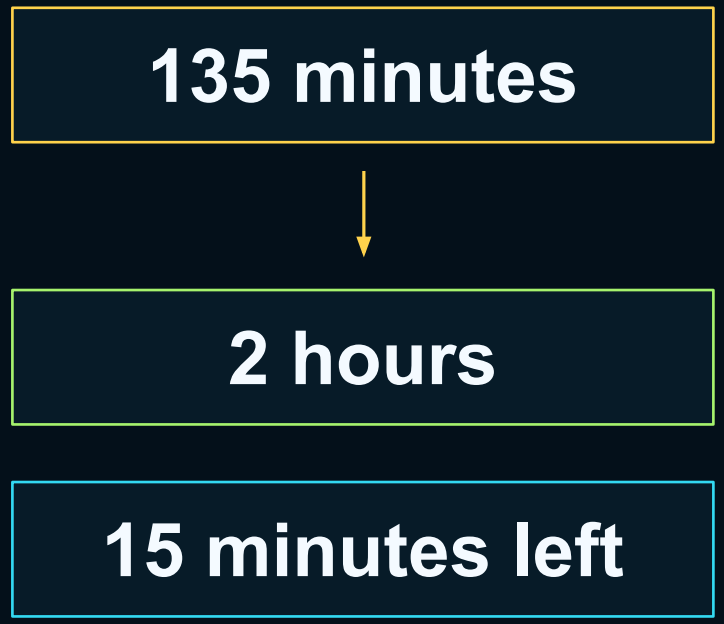
Lab 5: minutes to hours and remaining minutes

This is where integer division and remainder become practical.

```
total_minutes = 135

hours = total_minutes // 60
minutes = total_minutes % 60

print(hours)
print(minutes)
```



// gives the full hours. % gives what is left over after full hours are removed.

GUIDED EXERCISE

Lab 6: bytes to kilobytes and megabytes

Storage sizes connect back to the morning bits-and-bytes lesson.

```
file_size_bytes = 5242880

kilobytes = file_size_bytes / 1024
megabytes = kilobytes / 1024

print(kilobytes)
print(megabytes)
```

5,242,880 bytes

5,120.0 KB

5.0 MB

Computers count storage in powers of 2. 1024 is 2^{10}.

GUIDED EXERCISE

Predict before you run

Prediction builds the mental model faster than copy/paste.

```
a = 10
b = 3

print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

- Write predictions on paper first
- Then run the code
- Circle anything surprising
- Explain one line to a neighbor

ANSWER CHECK

Answer check: calculation outputs

Use this as a quick reveal after students attempt the lab.

Total cost: 100.0

Average: 86.33333333333333

Percent: 75.0

Kilometers: 8.0467

135 minutes: 2 and 15

Storage: 5120.0 KB, 5.0 MB

```
result = value_a operator value_b
```

The habit matters more than memorizing: name values, calculate, store, print, explain.

DISCUSSION

Common errors and quick fixes

Most calculation bugs come from small syntax, naming, or type issues.

NAME ERROR

totl_cost

The variable name is misspelled.

TYPE ERROR

"Total: " + 100

Text plus number needs help.

LOGIC ERROR

score1 + score2 / 2

Runs, but calculation is wrong.

Debugging habit: read the error, check spelling, check types, then check the math.

DISCUSSION

Discussion questions

Students should explain calculations in plain language.

What is an expression?

Why does Python use * for multiplication and ** for exponents?

When would you use /, //, and %?

Why do parentheses make code safer to read?

Why can a simple Python calculation require many lower-level operations?

Good answer style: “Python looks up the values, performs the operation, then stores or displays the result.”

WRAP-UP

Mini-lab checklist

By the end, students should be comfortable writing and running basic calculations.

1. **Define** variables holding numbers
2. **Calculate** with +, -, *, /, //, %, and **
3. **Store** the result in a new variable
4. **Predict** output before executing code
5. **Fix** simple type and naming errors
6. **Explain** how Python hides lower-level machine work

Next

Use expressions inside programs that make choices and process real data.